西电食堂美食点评系统

软件详细设计说明书

1. 引言

1.1 编写目的

本说明书旨在以严谨、详实的笔触，全方位阐述食堂点评系统的设计方案。从系统架构搭建、功能模块构建，到数据库结构设计等关键环节，均进行细致入微的描述，为系统的开发、测试、维护以及后续迭代提供精准且权威的指导依据。

1.2 项目背景

在当今数字化时代，食堂服务质量的提升与精细化管理成为重要需求。为搭建高效的沟通桥梁，促进食堂商家与就餐用户的良性互动，改善用户就餐体验，特开发本食堂点评系统。该系统集成用户点评、商家管理和系统管理等多元功能，通过数据驱动的方式，助力食堂服务质量的持续优化与管理效率的提升。

2. 系统架构设计

2.1 技术架构

系统采用成熟稳定的技术栈进行开发，具体如下：

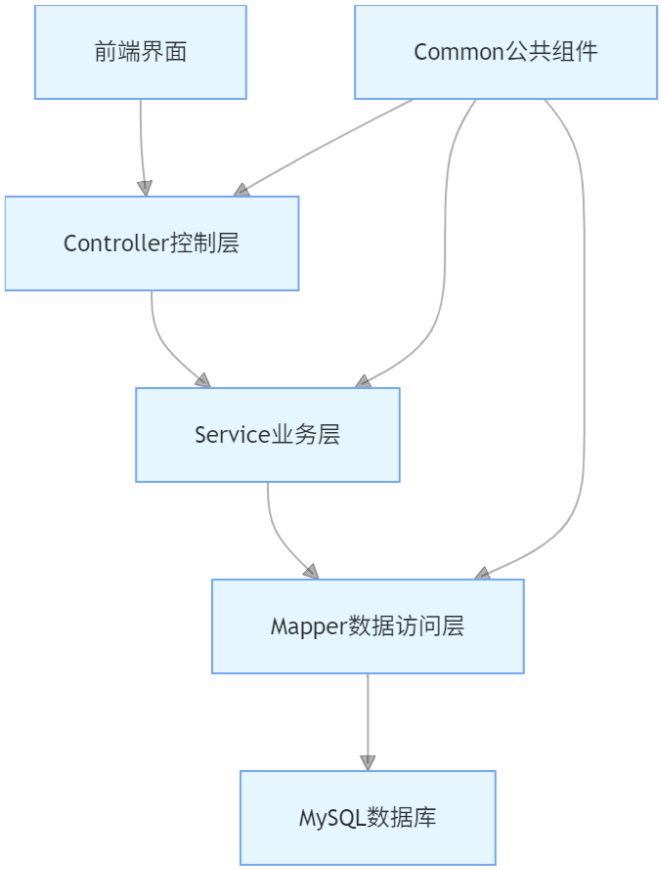
开发语言：Java，凭借其跨平台性、面向对象特性及丰富的类库，确保系统的高效开发与稳定运行。

框架：Spring Boot，通过简化配置和快速构建的优势，加速项目开发进程，提升开发效率。

数据库：MySQL，作为广泛应用的关系型数据库，能够高效存储和管理系统数据。

项目管理：Maven，实现项目依赖的自动化管理，便于团队协作与项目构建。

2.2 系统架构图



系统架构采用分层设计模式，前端界面负责与用户交互，接收用户请求并展示数据；Controller 控制层接收前端请求，调用 Service 业务层处理业务逻辑；Service 业务层执行业务处理，调用 Mapper 数据访问层进行数据操作；Mapper 数据访问层与 MySQL 数据库进行交互，实现数据的增删改查；Common 公共组件为各层提供通用的工具和方法，提升代码复用性。

2.3 目录结构

```
dianping/
├── src/main/java/com/example/sens/
│   ├── controller/    # 控制器层，处理前端请求与响应
│   ├── service/       # 业务逻辑层，实现具体业务功能
```

```
│   ├── mapper/        # 数据访问层，负责数据库操作
│   ├── entity/        # 实体类，对应数据库表结构
│   ├── common/        # 公共组件，包含通用工具类和常量
│   └── Application.java  # 启动类，用于启动 Spring Boot 应用
├── resources/         # 配置文件，存放系统配置信息
├── pom.xml            # 项目依赖管理文件，定义项目依赖库
└── dianping.sql       # 数据库脚本，用于创建和初始化数据库表
```

3. 功能模块设计

3.1 用户模块

3.1.1 菜品浏览

功能描述：为用户提供全面的食堂菜品信息浏览服务，使用户能够快速、便捷地了解食堂菜品情况。

主要接口：

获取菜品列表：根据不同条件获取食堂菜品的列表信息，如按食堂、按分类等。

获取菜品详情：获取指定菜品的详细信息，包括菜品名称、价格、介绍、图片等。

菜品搜索：支持用户通过菜品名称、关键词等进行搜索，快速定位所需菜品。

菜品分类查询：根据菜品分类，查询该分类下的所有菜品。

3.1.2 评价管理

功能描述：允许用户对食堂和菜品进行客观、真实的评价，促进食堂服务质量的提升。

主要接口：

发表评价：用户可以针对食堂的服务、环境、菜品口味等方面发表自己的评价和意见。

评分打分：用户对食堂和菜品进行评分，包括总评分、服务评分、环境评分、口味评分等。

评价回复：用户可以对其他用户的评论进行回复，形成良好的互动交流氛围。

评价列表查询：查询自己发表的评价列表，以及查看其他用户对食堂和菜品的评价。

3.1.3 个人中心

功能描述：实现用户个人信息及操作的集中管理，方便用户对自己的信息和操作进行

查看和调整。

主要功能：

收藏管理：用户可以对喜欢的菜品、店铺进行收藏，方便日后查看和选择。

评价记录：查看自己对食堂和菜品发表的所有评价记录。

个人信息设置：用户可以修改自己的个人信息，如昵称、头像、联系方式等。

笔记管理：用户可以记录自己的就餐心得、推荐菜品等笔记信息。

3.2 商户模块

3.2.1 店铺管理

1. **功能描述：**为商户提供便捷的店铺信息管理功能，帮助商户高效运营店铺。
2. **主要功能：**
    1) 店铺信息维护：商户可以对店铺的基本信息进行更新和完善，如店铺名称、地址、联系方式、店铺介绍等。
    2) 菜品管理：包括菜品的添加、编辑、删除、上下架等操作，方便商户根据实际情况调整菜品。
    3) 评价管理：商户可以查看用户对店铺和菜品的评价，及时回复用户的评价，处理用户反馈的问题。
    4) 数据统计：对店铺的经营数据进行统计分析，如菜品销量、用户评价数据、营业额等，为商户的经营决策提供数据支持。

3.2.2 菜品预告

1. **功能描述：**支持商户发布特价菜品和优惠信息，吸引用户就餐，提升店铺竞争力。
2. **主要功能：**
    1) 特价菜品发布：商户可以设置特价菜品的信息，包括菜品名称、原价、特价、活动时间等。
    2) 优惠套餐管理：创建和管理优惠套餐，如套餐名称、包含菜品、套餐价格、优惠力度等。
    3) 预告信息设置：设置菜品预告的展示时间、展示方式等，确保用户能够及时了解优惠信息。

3.3 管理员模块

3.3.1 系统管理

1. **功能描述**：对系统进行全面的管理和维护，保障系统的稳定运行和数据安全。
2. **主要功能**：
    1) 用户管理：对系统中的所有用户进行管理，包括用户信息的查看、编辑、删除，以及用户权限的分配和调整。
    2) 商铺管理：审核商铺的入驻申请，管理商铺信息，对违规商铺进行处理。
    3) 权限管理：设置不同角色的权限，确保用户只能访问和操作其权限范围内的功能和数据。
    4) 系统设置：对系统的基础配置进行管理，如系统名称、LOGO、公告信息等。

3.3.2 营销管理

1. **功能描述**：负责系统营销功能的策划和管理，提升系统的用户活跃度和食堂的影响力。
2. **主要功能**：
    1) 推荐管理：设置推荐菜品、推荐店铺，根据用户的浏览和消费记录进行个性化推荐。
    2) 榜单管理：生成和管理各类榜单，如热门菜品榜、好评店铺榜等，引导用户消费。
    3) 活动管理：创建和管理营销活动，如满减活动、抽奖活动等，吸引用户参与。

4. 数据库设计

4.1 数据库表设计

4.1.1 店铺信息表（biz_store）

```
CREATE TABLE `biz_store` (
    `id` bigint PRIMARY KEY,        -- 店铺 ID，唯一标识店铺
    `name` varchar(255),            -- 店铺名称
    `image` varchar(255),           -- 店铺照片，存储照片路径
```

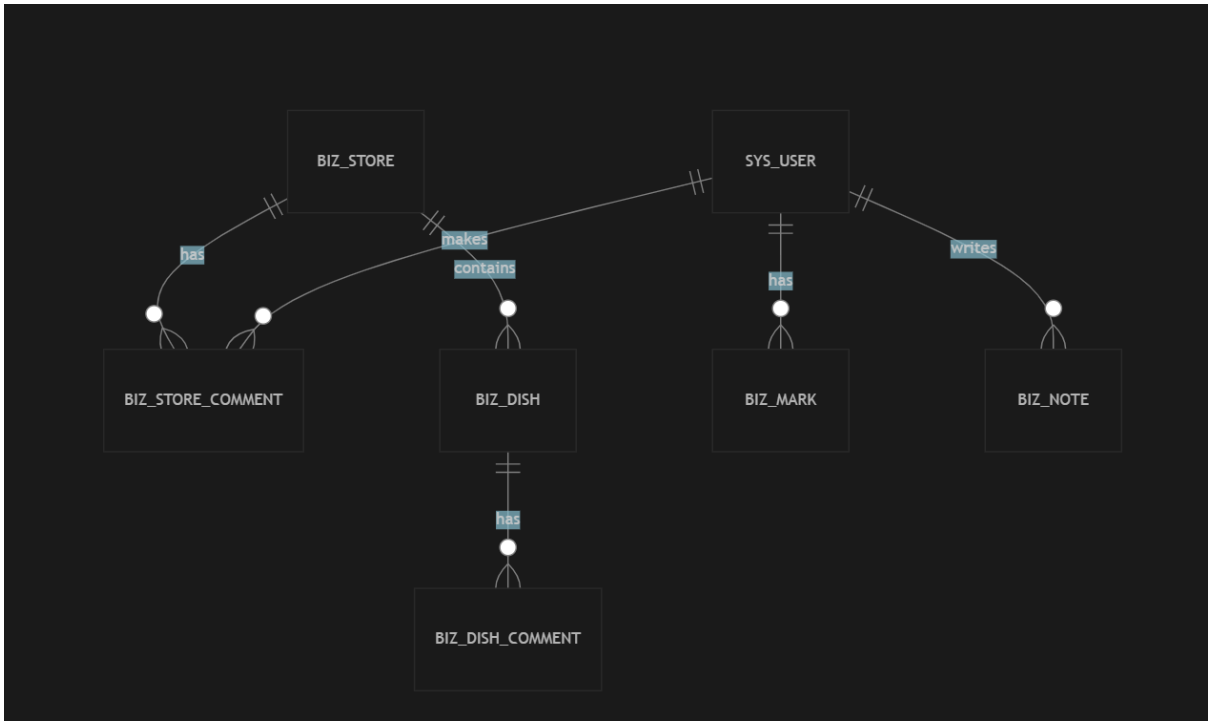
```
    `owner_name` varchar(255),      -- 责任人姓名
    `phone` varchar(255),           -- 联系电话
    `user_id` bigint,               -- 商家用户 ID，关联用户表
    `status` int,                   -- 状态，如启用、停用
    `views` int,                    -- 访问量，记录店铺被浏览次数
    `comments` int,                 -- 评论数，统计店铺收到的评论数量
    `score` decimal(2,1),           -- 总评分，综合各项评分
    `service_score` decimal(2,1),   -- 服务评分
    `environment_score` decimal(2,1), -- 环境评分
    `taste_score` decimal(2,1),     -- 口味评分
    `address` varchar(255),         -- 地址
    `sort` int,                     -- 排序，用于调整店铺展示顺序
    `canteen_id` int,               -- 食堂 ID，关联食堂表
    `is_valid` tinyint              -- 是否有效，标识店铺是否正常可用
)
```

4.1.2 评价表（biz_store_comment）

```
CREATE TABLE `biz_store_comment` (
    `id` bigint PRIMARY KEY,        -- 评论 ID，唯一标识评论
    `store_id` bigint,              -- 店铺 ID，关联店铺表
    `user_id` bigint,               -- 用户 ID，关联用户表
    `content` text,                 -- 评论内容
    `parent_id` bigint,             -- 父评论 ID，用于实现评论回复功能
    `score` decimal(2,1),           -- 总评分
    `service_score` decimal(2,1),   -- 服务评分
```

```
    `environment_score` decimal(2,1), -- 环境评分
    `taste_score` decimal(2,1),       -- 口味评分
    `status` int,                     -- 状态，如正常、审核中、已删除
    `create_time` datetime            -- 创建时间，记录评论发表时间
)
```

4.2 核心表关系



系统数据库通过外键关联构建核心表关系，如店铺信息表（biz_store）中的 user_id 关联用户表，用于标识店铺所属商家；评价表（biz_store_comment）中的 store_id 和 user_id 分别关联店铺信息表和用户表，明确评论的对象和发表者。通过合理的表关系设计，确保数据的一致性和完整性，为系统功能的实现提供坚实的数据基础。

5. 接口设计

5.1 RESTful API 设计规范

1. **使用 HTTP 动词表示操作：**通过 GET 获取资源、POST 创建资源、PUT 更新资源、DELETE 删除资源，清晰定义接口操作语义。
2. **使用复数名词表示资源：**以统一的命名规范表示资源集合，增强接口的可读性和

可维护性。

3. **使用嵌套资源表示从属关系**：对于具有从属关系的资源，采用嵌套资源的方式进行表示，准确反映资源之间的逻辑关系。

5.2 主要接口示例

```
// 店铺相关接口
GET    /api/stores            // 获取店铺列表，支持分页、筛选等参数
POST   /api/stores            // 创建店铺，请求体包含店铺信息
GET    /api/stores/{id}       // 获取店铺详情，{id}为店铺 ID
PUT    /api/stores/{id}       // 更新店铺信息，{id}为店铺 ID，请求体包含更新内容
DELETE /api/stores/{id}       // 删除店铺，{id}为店铺 ID
// 评论相关接口
GET    /api/comments          // 获取评论列表，支持按店铺、用户等条件筛选
POST   /api/comments          // 创建评论，请求体包含评论内容
PUT    /api/comments/{id}     // 更新评论，{id}为评论 ID，请求体包含更新内容
DELETE /api/comments/{id}     // 删除评论，{id}为评论 ID
```

6. 安全设计

6.1 权限控制

采用基于角色的访问控制（RBAC）模型，定义普通用户、商户、管理员三种用户角色，并对权限进行细粒度划分：

1. **菜单权限**：控制用户可访问的功能菜单。
2. **按钮权限**：针对菜单内的具体操作按钮，设置不同用户的操作权限。
3. **数据权限**：限制用户对数据的访问范围，如商户只能查看和管理自己店铺的数据，管理员可查看所有数据。

6.2 数据安全

1. **用户密码加密存储**：使用安全的加密算法对用户密码进行加密存储，防止密码泄露。

2. **敏感数据加密传输**：在数据传输过程中，对敏感数据如用户个人信息、支付信息等进行加密处理，确保数据安全。
3. **SQL 注入防护**：采用参数化查询、输入验证等方式，防止 SQL 注入攻击，保护数据库安全。
4. **XSS 防护**：对用户输入进行过滤和转义，防止跨站脚本攻击（XSS），保障用户和系统安全。

7. 性能设计

7.1 优化措施

1. **数据库索引优化**：根据业务需求，为数据库表的常用查询字段添加索引，提高数据查询效率。
2. **缓存机制**：引入缓存技术，对热点数据进行缓存，减少数据库查询压力，提升系统响应速度。
3. **图片存储优化**：采用合适的图片存储方案，如图片压缩、CDN 分发等，降低图片存储和传输成本，提高页面加载速度。
4. **分页查询**：对数据量较大的查询结果进行分页处理，避免一次性加载过多数据，提升用户体验。

7.2 并发处理

1. **数据库连接池**：使用数据库连接池管理数据库连接，提高数据库连接的复用率，减少连接创建和销毁的开销。
2. **线程池管理**：通过线程池合理分配和管理线程资源，提高系统的并发处理能力。
3. **并发控制**：采用乐观锁、悲观锁等并发控制机制，确保在高并发场景下数据的一致性和完整性。

8. 部署方案

8.1 系统要求

- JDK：1.8 及以上版本，提供系统运行所需的 Java 环境。
- MySQL：5.7 及以上版本，满足数据库存储和管理需求。
- Maven：3.6 及以上版本，用于项目的构建和依赖管理。

8.2 部署步骤

1. **数据库初始化**：执行 dianping.sql 数据库脚本，创建和初始化系统所需的数据库表结构和基础数据。
2. **配置文件修改**：根据实际部署环境，修改 resources 目录下的配置文件，如数据库连接配置、系统参数配置等。
3. **打包部署**：使用 Maven 命令对项目进行打包，生成可执行的 JAR 包，并将其部署到服务器指定目录。
4. **服务启动**：在服务器上运行 JAR 包，启动食堂点评系统服务，确保系统正常运行。

9. 扩展性设计

9.1 可扩展点

1. **评分规则扩展**：支持根据不同需求灵活调整评分规则，如增加新的评分维度、修改评分权重等。
2. **榜单算法扩展**：方便对榜单生成算法进行优化和扩展，以提供更准确、更具吸引力的榜单。
3. **营销活动扩展**：快速添加新的营销活动类型，如团购活动、限时折扣活动等，丰富系统营销功能。
4. **支付功能扩展**：预留接口，便于后续集成第三方支付功能，实现线上支付。

9.2 代码设计

1. **接口抽象**：通过定义抽象接口，将不同功能模块的实现细节进行封装，提高代码的可维护性和扩展性。
2. **设计模式应用**：运用常见的设计模式，如工厂模式、单例模式、策略模式等，优化代码结构，提升代码质量。
3. **模块化设计**：将系统划分为多个功能模块，每个模块具有独立的功能和清晰的接口，降低模块之间的耦合度，方便模块的独立开发、测试和维护。